SC4023 Big Data Management
Group Project

Group number: 6

Name	Matriculation No.
Ong Yao Sheng	U2322398H
Liew Wei Jie	U232119H
Chin Shao Yang	U2321398E

TABLE OF CONTENTS

1 Data Storage **3**
1.1 System Architecture 3
1.2 Data Type Integrity 3
1.3 Compression Town 3
1.4 Precompute Price Per Square Metre 3
1.5 Data encoding date 3
1.6 Pre-sorting 4
1.7 Exception Handling 4
2 Data Processing **4**
2.1 Scanning columns according to task conditions 4
2.2 Determining and retrieving matched data 4
2.3 Improving efficiency in scanning columns and reusing intermediate results 4
3 Experiment Results **5**
Appendix **8**

1 Data Storage

1.1 System Architecture

The system begins by prompting the user for a raw Resale CSV file path. Although it supports multiple storage formats, this implementation is optimised for a column-oriented structure. During ingestion, the CSV is parsed and vertically partitioned based on the header, with each attribute stored as a separate .col file (e.g., month.col, floor_area.col, town_int.col) inside Database/<db_name>/. This ensures only the required columns are read during query execution, reducing I/O overhead. After setup, the user selects the database and provides their matriculation number to run queries. The engine filters data based on the target year, months, and matched towns, producing 568 final rows, which are exported as a CSV in the result directory. Input reading: The raw CSV is read using CSVLoader (utils/csv_loader.py), which calls pandas.read_csv() on the user-supplied file path. The resulting DataFrame is validated for existence and passed to ColumnFormat.write() for ingestion. An lru_cache decorator ensures the file is only read once per process if called multiple times. Output writing: After all (x, y) pairs are evaluated, OutputWriter (utils/output_writer.py) creates result/ScanResult_<MatricNum>.csv using Python's built-in csv.writer. The header row is written first, followed by one row per (x, y) pair. If no qualifying record exists, the row contains "No result" in place of record fields. The month_num integer is split back into YYYY and MM components for output, and psm_price is rounded to an integer as required.

1.2 Data Type Integrity

When creating .col files from CSV data, we map each field to its correct data type to ensure data integrity and efficient storage. Types such as Int, Float, or Date keep values consistent and accurate, while also using less space than strings, which require extra bytes for characters and metadata.

1.3 Compression Town

To reduce storage and improve query performance, categorical fields such as town names are encoded using dictionary encoding. Each unique town name is mapped to an integer ID, replacing repeated strings with compact numeric codes. This reduces storage usage from 259k * 8 bytes → 259k * 4 bytes (>50%) and speeds up query execution, as numerical comparisons are faster than string comparisons for filtering and equality checks.

1.4 Precompute Price Per Square Metre

Price per square metre is pre-computed from Resale_price / Floor_area during database creation and stored as an additional column to reduce query-time computation. Instead of recalculating this value repeatedly, the query engine simply scans the stored column, reducing processing overhead and improving query responsiveness.

1.5 Data encoding date

To improve filtering efficiency, temporal attributes like month and year are converted from strings into integer encodings during database creation. For example, a value like Jan-15 is transformed into an integer such as 201501, enabling faster numerical comparisons and more efficient query execution.

1.6 Pre-sorting

During database creation, after derived columns are generated, the engine pre-sorts data by month_num before writing the .col files. This enables more efficient month-based range queries, as the ordered structure allows the query engine to narrow the search space and avoid scanning irrelevant records, improving overall query performance.

1.7 Exception Handling

Empty results are represented as empty lists / None, detected by controller logic, and written out as "No result" in the CSV.

2 Data Processing

2.1 Scanning columns according to task conditions

When a query is initiated, the Query object (model/QueryModel.py) scans all column files once into an in-memory _column_cache dictionary at initialization. This cache maps each column name to its full list of values. All subsequent filtering operations read exclusively from this cache; no further disk I/O occurs during a query.

The scan is driven by the task conditions derived from the matriculation number: a target year and starting month (e.g., from U2321398E → year 2018, month 03), and a set of matched towns (CHOA CHU KANG, HOUGANG, WOODLANDS, TAMPINES, BUKIT PANJANG, YISHUN). The query engine then iterates over all valid (x, y) pairs where $x \in [1, 8]$ (month window) and $y \in [80, 150]$ (minimum floor area in m²).

2.2 Determining and retrieving matched data

The query engine maintains a row index list (_selected_indexes) representing the currently qualifying rows. Filters are applied sequentially, each narrowing this index set:

- Town filter: where_in("town_int", valid_towns): filters to rows matching the valid towns
- Time lower bound: where_gte("month_num", start_month): retains rows on or after the starting month
- Time upper bound: where_lte("month_num", end_month): retains rows within the x-month window
- Floor area filter: where_gte("floor_area_sqm", y): retains rows with floor area ≥ y m²

After all filters are applied, the engine calls aggregate("psm_price", "min") — a single-pass generator over the surviving index set — to find the minimum price per square meter. If this minimum is ≤ 4725, the (x, y) pair is considered valid. A where_eq on that minimum value then isolates the matching record, and fetch() materialises the full row from the column cache.

2.3 Improving efficiency in scanning columns and reusing intermediate results

Several strategies are used to avoid redundant computation across the 568 (x, y) combinations:

Clone-based result reuse: After applying the town filter and start-month lower bound once, a base_query object is created. For each value of x, this base is cloned, sharing the same _column_cache in memory but

copying only the current index list. The upper-month filter is then applied to the clone. This avoids re-running the town and start-month filters for every x.

Bitmap index for town filtering: The town_int column has a prebuilt bitmap index (stored in metadata). Each unique town is represented as an integer bitset, where bit i is set if row i belongs to that town. The where_in operation performs bitwise OR across the relevant town bitmaps, then ANDs the result with the current selection, making multi-town filtering a set operation rather than a per-row loop. However, this method shows a slower time.

Binary search for month-range filtering: Because the data is pre-sorted by month_num at storage time (ColumnFormat.sort_column), the where_gte and where_lte operations can use bisect_left / bisect_right to identify the exact start and end indices in O(log n) instead of a full O(n) linear scan.

Zone map for block skipping: A zone map stores the minimum and maximum month_num for each fixed-size block of rows. Before binary search, the engine uses ZoneMap.find_start to skip entire blocks whose maximum value is below the query threshold, reducing the number of rows to examine.

Memory-efficient aggregation: The aggregate function uses Python generators over the surviving index list, computing min in a single pass without materialising a new list of filtered values.

Metadata to store database information: A metadata file improves performance and reliability by storing schema, data types, storage engine, sorting order, and indexes. This avoids runtime re-inference, enabling faster loading and consistent type handling. It also improves query planning by allowing immediate use of optimizations like indexes or binary search instead of full scans, reducing overhead and improving execution speed.

Selective filtering: Improves query performance by applying the most restrictive condition first to reduce intermediate results early. The system estimates selectivity and prioritises the filter that produces the fewest rows. For example, filtering by town_int reduces 259,237 rows to 62,663, while month_num reduces it to 240,177 rows. Applying town_int first significantly shrinks the dataset before applying month_num, whereas doing the reverse results in less reduction and more processing. This reduces I/O and overall execution cost.

3 Experiment Results

The program was run using the database ResalePrices and the matriculation number U2321398E. The program loaded 13-column files from ResalePrices.csv and completed the query in 0.1160 seconds. The result was then output as result/ScanResult_U2321398E.csv.

The resulting output follows the required schema with header row (x, y), Year, Month, Town, Block, Floor_Area, Flat_Model, Lease_Commence_Date, Price_Per_Square_Meter, followed by rows ordered by increasing x and then increasing y. Price_Per_Square_Meter values are rounded to integers as specified.

```
PS C:\Users\chins\OneDrive\Desktop\SC4023\SC4023-BIG-DATA-MANAGEMENT> python -c "import itertools; print(''.join(itertools.islice(
open(r'result/ScanResult_U2321398E.csv','r',encoding='utf-8'),8)))"
"(x, y)",Year,Month,Town,Block,Floor_Area,Flat_Model,Lease_Commence_Date,Price_Per_Square_Meter
"(1, 80)",2018,09,WOODLANDS,10,126.0,Standard,1976,2563
"(1, 81)",2018,09,WOODLANDS,10,126.0,Standard,1976,2563
"(1, 82)",2018,09,WOODLANDS,10,126.0,Standard,1976,2563
"(1, 83)",2018,09,WOODLANDS,10,126.0,Standard,1976,2563
"(1, 84)",2018,09,WOODLANDS,10,126.0,Standard,1976,2563
"(1, 85)",2018,09,WOODLANDS,10,126.0,Standard,1976,2563
"(1, 86)",2018,09,WOODLANDS,10,126.0,Standard,1976,2563
```

Fig 6, Verification of output format

The scan result file reports the record with the minimum price per square meter (rounded) for (1,80): Year 2018, Month 09, Town WOODLANDS, Block 10, Floor_Area 126, Flat_Model Standard, Lease_Commence_Date 1976, and Price_Per_Square_Meter = 2563 as per verification done using Excel.

A	B	C	D	E	F	G	H	I	J	K
(x, y)	Year	Month	Town	Block	Floor_Area	Flat_Model	Lease_Com	Price_Per_Square_Meter		
(1, 80)	2018	9	WOODLAN	10	126	Standard	1976	2563		

Fig 7, Result verification for (x,y) where x=1 and y=80

For the 7-month window and area threshold 150m^2, the output contains the matched record with the minimum rounded price per square meter: Year 2018, Month 10, Town CHOA CHU KANG, Block 108, Floor_Area 150, Flat_Model Maisonette, Lease_Commence_Date 1989, and Price_Per_Square_Meter = 3233.

497	(7, 149)	2018	10	CHOA CHU	108	150	Maisonette	1989	3233
498	(7, 150)	2018	10	CHOA CHU	108	150	Maisonette	1989	3233
499	(8, 80)	2018	9	WOODLAN	10	126	Standard	1976	2563

Fig 8, Result verification for (x,y) where x=7 and y=150

For the 8-month window and area threshold 145m^2, the output contains the matched record with the minimum rounded price per square meter: Year 2018, Month 10, Town CHOA CHU KANG, Block 527, Floor_Area 148, Flat_Model Apartment, Lease_Commence_Date 1995, and Price_Per_Square_Meter = 3209.

563	(8, 144)	2018	10	CHOA CHU	681	144	Premium A	1999	3181
564	(8, 145)	2018	10	CHOA CHU	527	148	Apartment	1995	3209
565	(8, 146)	2018	10	CHOA CHU	527	148	Apartment	1995	3209

Fig 9, Result verification for (x,y) where x=8 and y=145

To verify correctness and evaluate performance differences, the query was executed on both the unsorted and pre-sorted datasets. In both cases, the program successfully loaded 13 columns and applied the same filtering sequence, including the town filter (reducing the dataset to 62,663 rows) followed by the month

filter (reducing it further to 45,819 rows). This confirms that the query logic is consistent across both datasets and produces identical intermediate results.

On the unsorted dataset (Database\linear), the query completed in 0.1160 seconds. This execution reflects a full evaluation of the dataset without any ordering-based optimisations.

In contrast, execution on the dataset sorted by month_num (Database\sortmth) completed in 0.1055 seconds. The reduction in runtime demonstrates the effectiveness of pre-sorting, as it enables more efficient month-range filtering by allowing the query engine to perform faster boundary-based lookups instead of scanning the full dataset.

Overall, both runs confirm functional correctness, while the performance improvement observed in the sorted dataset validates the optimisation introduced through pre-sorting.

Appendix

```python
for x in range(1, 9):
    end_month = Helpers.add_months(start_yr_mth, int(x))
    base_query.where_lte("month_num", end_month)

    area_query = base_query.clone()

    for y in range(80, 151):
        area_query.where_gte("floor_area_sqm", float(y))

        min_psm = area_query.aggregate("psm_price", "min")

        if min_psm is None or min_psm > 4725:
            results.append({"x": x, "y": y, "row": None})
            continue

        q = area_query.clone()
        q.where_eq("psm_price", min_psm)

        flats = q.fetch()

        if not flats:
            results.append({"x": x, "y": y, "row": None})
        else:
            results.append({"x": x, "y": y, "row": flats[0]})
```

Fig 1, Implementation of query engine.

```
Number of records after month filter: 240177
Number of records after town filter: 12309

Query completed in 0.2650 seconds.
```

Fig 2, Intermediate results filtering by month first

```
Number of records after town filter: 13451
Number of records after month filter: 12309

Query completed in 0.1204 seconds.
```

Fig 3, Intermediate results filtering by town first

```python
try:
    local_idx = bisect.bisect_left(col_data, threshold, lo=block_start, hi=block_end)
    # adjust to be relative to the block
    local_idx = local_idx - block_start
except TypeError:
    slice_strs = [str(x) for x in col_data[block_start:block_end]]
    local_idx = bisect.bisect_left(slice_strs, str(threshold))
```

Fig 4, Implementation of binary search on filtering condition.

```
PS C:\Users\chins\OneDrive\Desktop\SC4023\SC4023-BIG-DATA-MANAGEMENT> python main.py

=== Big Data Management System ===
1. Load existing database
2. Create new database
3. Exit

Enter your choice: 1

Available databases:
1. ResalePrices
2. sortedmth

Enter database name or number: 1

Enter matric number: U2321398E
[ColumnFormat] Read column 'block' (259237 rows)
[ColumnFormat] Read column 'flat_model' (259237 rows)
[ColumnFormat] Read column 'flat_type' (259237 rows)
[ColumnFormat] Read column 'floor_area_sqm' (259237 rows)
[ColumnFormat] Read column 'lease_commence_date' (259237 rows)
[ColumnFormat] Read column 'month' (259237 rows)
[ColumnFormat] Read column 'month_num' (259237 rows)
[ColumnFormat] Read column 'psm_price' (259237 rows)
[ColumnFormat] Read column 'resale_price' (259237 rows)
[ColumnFormat] Read column 'storey_range' (259237 rows)
[ColumnFormat] Read column 'street_name' (259237 rows)
[ColumnFormat] Read column 'town' (259237 rows)
[ColumnFormat] Read column 'town_int' (259237 rows)
[ColumnFormat] read loaded 13 columns from 'Database\ResalePrices'
Number of records after town filter: 62663
Number of records after month filter: 45819

Query completed in 0.1160 seconds.
Results written to result/ScanResult_U2321398E.csv
```

Fig 5, Program successfully ran, and results outputted

CONTRIBUTION FORM

Group 6

Name	Matriculation Number	Detailed Individual Contribution	Percentage (100% in total)
Ong Yao Sheng	U2322398H	Discussed and written code and report	33.33%
Liew Wei Jie	U2321169H	Discussed and written code and report	33.33%
Chin Shao Yang	U2321398E	Discussed and written code and report	33.33%

Name and Signature from all group members:

Name and Signature of Member 1

Liew Wei Jie,

Name and Signature of Member 2

Ong Yao Sheng,

Name and Signature of Member 3

Chin Shao Yang,